In [1]:
```
import sys, subprocess

subprocess.check_call([sys.executable, "-m", "pip", "uninstall", "-y", "jax"

subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "--forc
    "numpy==1.24.3",
    "tensorflow==2.15.0",
    "tensorflow-data-validation==1.15.1",
    "tensorflow-metadata==1.15.0",
    "tfx-bsl==1.15.1",
    "apache-beam==2.53.0",
    "pandas==1.5.3",
    "pyarrow==10.0.1"
])

print("Done. Restart runtime now.")
```

```
Done. Restart runtime now.
```

In [1]:
```
import tensorflow as tf
import tensorflow_data_validation as tfdv
print(tf.__version__, tfdv.__version__)
```

```
2.15.0 1.15.1
```

1 - Setup and Imports

In [3]:
```
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import tensorflow_data_validation as tfdv
import pyarrow as pa

print("TF:", tf.__version__)
print("TFDV:", tfdv.__version__)
print("Pandas:", pd.__version__)
print("PyArrow:", pa.__version__)

os.makedirs("data", exist_ok=True)
os.makedirs("output", exist_ok=True)
```

```
TF: 2.15.0
TFDV: 1.15.1
Pandas: 1.5.3
PyArrow: 10.0.1
```

2 - Load the Dataset

In [4]:
```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult
columns = [
```

```
    "age", "workclass", "fnlwgt", "education", "education_num", "marital_sta
    "occupation", "relationship", "race", "sex", "capital_gain", "capital_lo
    "hours_per_week", "native_country", "income"
]

df = pd.read_csv(
    url,
    header=None,
    names=columns,
    skipinitialspace=True,
    na_values=["?", " ?"]
).dropna().reset_index(drop=True)

df.to_csv("data/adult_income.csv", index=False)
print("Cleaned shape:", df.shape)
df.head()
```

```
Cleaned shape: (30162, 15)
```

Out[4]:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	re
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	

2.1 Read and Split the Dataset

We split the cleaned dataset into training, evaluation, and serving sets.

Data splits

In [5]:

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(df))

n = len(df)
n_train = int(0.70 * n)
n_eval = int(0.15 * n)

train_df = df.iloc[idx[:n_train]].copy()
eval_df = df.iloc[idx[n_train:n_train+n_eval]].copy()
serving_df = df.iloc[idx[n_train+n_eval:]].copy()

print("Before label drop:", train_df.shape, eval_df.shape, serving_df.shape)
```

```
Before label drop: (21113, 15) (4524, 15) (4525, 15)
```

Label Column

Serving data should not include the label (income), so we remove it. We also remove an irrelevant feature (fnlwgt) from all splits.

In [6]:
```
serving_df = serving_df.drop(columns=["income"])

# Remove irrelevant feature
drop_cols = ["fnlwgt"]
train_df = train_df.drop(columns=drop_cols)
eval_df = eval_df.drop(columns=drop_cols)
serving_df = serving_df.drop(columns=drop_cols)

train_df.to_csv("data/train.csv", index=False)
eval_df.to_csv("data/eval.csv", index=False)
serving_df.to_csv("data/serving.csv", index=False)

print("Final splits:", train_df.shape, eval_df.shape, serving_df.shape)
print("Serving columns contain income?", "income" in serving_df.columns)
```

```
Final splits: (21113, 14) (4524, 14) (4525, 13)
Serving columns contain income? False
```

3 - Generate and Visualize Training Data Statistics

Removing Irrelevant Features

`fnlwgt` was removed from all splits.

Generate Training Statistics

In [7]:
```
stats_options = tfdv.StatsOptions(feature_allowlist=list(train_df.columns))
train_stats = tfdv.generate_statistics_from_csv("data/train.csv", stats_opti
```

```
WARNING:apache_beam.runners.interactive.interactive_environment:Dependencies
required for Interactive Beam PCollection visualization are not available, p
lease use: `pip install apache-beam[interactive]` to install necessary depen
dencies to enable all data visualization features.
WARNING:apache_beam.io.tfrecordio:Couldn't find python-snappy so the impleme
ntation of _TFRecordUtil._masked_crc32c is not as fast as it could be.
WARNING:tensorflow:From /usr/local/lib/python3.11/dist-packages/tensorflow_d
ata_validation/utils/artifacts_io_impl.py:93: tf_record_iterator (from tenso
rflow.python.lib.io.tf_record) is deprecated and will be removed in a future
version.
Instructions for updating:
Use eager execution and:
`tf.data.TFRecordDataset(path)`
```

Visualize Training Statistics

In [8]:
```
tfdv.visualize_statistics(train_stats)
```

Sort by

Feature order

Reverse order

Feature search (regex enabled)

Features: int(5) string(9)

Numeric Features (5)

	count	missing	mean	std dev	zeros	min	median
age	21.1k	0%	38.43	13.15	0%	17	37
education_num	21.1k	0%	10.12	2.56	0%	1	10
capital_gain	21.1k	0%	1,159.55	7,748.31	91.49%	0	0
capital_loss	21.1k	0%	88.83	405.03	95.24%	0	0

4 - Infer a data schema

Infer the training set schema

In [9]:
```
schema = tfdv.infer_schema(train_stats)
tfdv.display_schema(schema)
```

Feature name	Type	Presence	Valency	Domain
'age'	INT	required		-
'workclass'	STRING	required		'workclass'
'education'	STRING	required		'education'
'education_num'	INT	required		-
'marital_status'	STRING	required		'marital_status'
'occupation'	STRING	required		'occupation'
'relationship'	STRING	required		'relationship'
'race'	STRING	required		'race'
'sex'	STRING	required		'sex'
'capital_gain'	INT	required		-
'capital_loss'	INT	required		-
'hours_per_week'	INT	required		-
'native_country'	STRING	required		'native_country'
'income'	STRING	required		'income'

Domain	Values
'workclass'	'Federal-gov', 'Local-gov', 'Private', 'Self-emp-inc', 'Self-emp-not-inc', 'State-gov', 'Without-pay'
'education'	'10th', '11th', '12th', '1st-4th', '5th-6th', '7th-8th', '9th', 'Assoc-acdm', 'Assoc-voc', 'Bachelors', 'Doctorate', 'HS-grad', 'Masters', 'Preschool', 'Prof-school', 'Some-college'
'marital_status'	'Divorced', 'Married-AF-spouse', 'Married-civ-spouse', 'Married-spouse-absent', 'Never-married', 'Separated', 'Widowed'
'occupation'	'Adm-clerical', 'Armed-Forces', 'Craft-repair', 'Exec-managerial', 'Farming-fishing', 'Handlers-cleaners', 'Machine-op-inspct', 'Other-service', 'Priv-house-serv', 'Prof-specialty', 'Protective-serv', 'Sales', 'Tech-support', 'Transport-moving'
'relationship'	'Husband', 'Not-in-family', 'Other-relative', 'Own-child', 'Unmarried', 'Wife'
'race'	'Amer-Indian-Eskimo', 'Asian-Pac-Islander', 'Black', 'Other', 'White'
'sex'	'Female', 'Male'
'native_country'	'Cambodia', 'Canada', 'China', 'Columbia', 'Cuba', 'Dominican-Republic', 'Ecuador', 'El-Salvador', 'England', 'France', 'Germany', 'Greece', 'Guatemala', 'Haiti', 'Holand-Netherlands', 'Honduras', 'Hong', 'Hungary', 'India', 'Iran', 'Ireland', 'Italy', 'Jamaica', 'Japan', 'Laos', 'Mexico', 'Nicaragua', 'Outlying-US(Guam-USVI-etc)', 'Peru', 'Philippines', 'Poland', 'Portugal', 'Puerto-Rico', 'Scotland', 'South', 'Taiwan', 'Thailand', 'Trinadad&Tobago', 'United-States', 'Vietnam', 'Yugoslavia'
'income'	'<=50K', '>50K'

5 - Calculate, Visualize and Fix Evaluation Anomalies

Compare Training and Evaluation Statistics

In [10]:
```
eval_stats = tfdv.generate_statistics_from_csv("data/eval.csv", stats_optior
tfdv.visualize_statistics(
    lhs_statistics=train_stats,
    rhs_statistics=eval_stats,
    lhs_name="TRAIN",
    rhs_name="EVAL"
)
```

Sort by

Feature order

Reverse order

Feature search (regex enabled)

Features: int(5) string(9)

TRAIN EVAL

Numeric Features (5)

	count	missing	mean	std dev	zeros	min	median
age							
	21.1k	0%	38.43	13.15	0%	17	37
	4,524	0%	38.67	13.26	0%	17	37
education_num							
	21.1k	0%	10.12	2.56	0%	1	10
	4,524	0%	10.14	2.52	0%	1	10
capital_gain							
	21.1k	0%	1,159.55	7,748.31	**91.49%**	0	0
	4,524	0%	900.08	6,506.41	**92%**	0	0
capital_loss							
	21.1k	0%	88.83	405.03	**95.24%**	0	0

Detecting Anomalies

In [11]:
```
eval_anomalies = tfdv.validate_statistics(
    statistics=eval_stats,
    schema=schema
)
tfdv.display_anomalies(eval_anomalies)
```

No anomalies found.

Fix evaluation anomalies in the schema

Even if eval shows no anomalies, we harden schema by setting explicit `income` domain.

In [12]:
```
if not any(sd.name == "income_domain" for sd in schema.string_domain):
    sd = schema.string_domain.add()
    sd.name = "income_domain"
    sd.value.append("<=50K")
    sd.value.append(">50K")

income_feature = tfdv.get_feature(schema, "income")
income_feature.domain = "income_domain"
```

```
eval_anomalies_after_fix = tfdv.validate_statistics(statistics=eval_stats, s
tfdv.display_anomalies(eval_anomalies_after_fix)
```

No anomalies found.

6 - Schema Environments

Check anomalies in the serving set

In [13]:
```
serving_stats = tfdv.generate_statistics_from_csv(
    data_location="data/serving.csv",
    stats_options=stats_options
)

serving_anomalies_before_env = tfdv.validate_statistics(
    statistics=serving_stats,
    schema=schema
)
tfdv.display_anomalies(serving_anomalies_before_env)
```

Feature name	Anomaly short description	Anomaly long description
'income'	Column dropped	Column is completely missing

Modifying the Domain

In [14]:
```
if "TRAINING" not in schema.default_environment:
    schema.default_environment.append("TRAINING")
if "SERVING" not in schema.default_environment:
    schema.default_environment.append("SERVING")

income_feature = tfdv.get_feature(schema, "income")
if "SERVING" not in income_feature.not_in_environment:
    income_feature.not_in_environment.append("SERVING")
```

Detecting anomalies with environments

In [15]:
```
serving_anomalies_after_env = tfdv.validate_statistics(
    statistics=serving_stats,
    schema=schema,
    environment="SERVING"
)
tfdv.display_anomalies(serving_anomalies_after_env)
```

No anomalies found.

7 - Check for Data Drift and Skew

In [16]:
```
for f in ["age", "education_num", "hours_per_week", "capital_gain", "capital
    feature = tfdv.get_feature(schema, f)
    feature.skew_comparator.infinity_norm.threshold = 0.03
    feature.drift_comparator.jensen_shannon_divergence.threshold = 0.10
```

In [17]:
```
# Skew: TRAIN vs SERVING
skew_anomalies = tfdv.validate_statistics(
    statistics=train_stats,
    schema=schema,
    serving_statistics=serving_stats,
    environment="TRAINING"
)
tfdv.display_anomalies(skew_anomalies)

# Drift: SERVING vs TRAIN baseline
drift_anomalies = tfdv.validate_statistics(
    statistics=serving_stats,
    schema=schema,
    previous_statistics=train_stats,
    environment="SERVING"
)
tfdv.display_anomalies(drift_anomalies)
```

No anomalies found.

No anomalies found.

Inject synthetic shift to force detectable drift/skew

In [21]:
```
serving_bad_df = serving_df.copy()
serving_bad_df["hours_per_week"] = (serving_bad_df["hours_per_week"] + 20).c
serving_bad_df["capital_gain"] = serving_bad_df["capital_gain"] * 3
serving_bad_df.loc[serving_bad_df.sample(frac=0.25, random_state=10).index,
serving_bad_df.to_csv("data/serving_bad.csv", index=False)

serving_bad_stats = tfdv.generate_statistics_from_csv("data/serving_bad.csv"

skew_anomalies_bad = tfdv.validate_statistics(
    statistics=train_stats,
    schema=schema,
    serving_statistics=serving_bad_stats,
    environment="TRAINING"
)
tfdv.display_anomalies(skew_anomalies_bad)

drift_anomalies_bad = tfdv.validate_statistics(
    statistics=serving_bad_stats,
    schema=schema,
    previous_statistics=train_stats,
    environment="SERVING"
)
tfdv.display_anomalies(drift_anomalies_bad)
```

No anomalies found.

Feature name	Anomaly short description	Anomaly long description
'hours_per_week'	High approximate Jensen-Shannon divergence between current and previous	The approximate Jensen-Shannon divergence between current and previous is 0.516291 (up to six significant digits), above the threshold 0.1.
'capital_gain'	High approximate Jensen-Shannon divergence between current and previous	The approximate Jensen-Shannon divergence between current and previous is 0.411142 (up to six significant digits), above the threshold 0.1.

In [28]:
```
slice_sample = train_df.sample(n=500, random_state=42)
slice_sample.to_csv("slice_sample.csv", index=False)
print("Saved slice_sample.csv:", slice_sample.shape)
```

```
Saved slice_sample.csv: (500, 14)
```

8 - Display Stats for Data Slices

In [29]:
```
slice_fn = tfdv.experimental_get_feature_value_slicer(features={"occupation"

sliced_stats = tfdv.generate_statistics_from_csv(
    data_location="slice_sample.csv",
    stats_options=tfdv.StatsOptions(
        feature_allowlist=list(train_df.columns),
        slice_functions=[slice_fn]
    )
)

tfdv.visualize_statistics(sliced_stats)
```

Sort by

Feature order

Reverse order

Feature search (regex enabled)

Features: int(5) string(9)

Numeric Features (5)

	count	missing	mean	std dev	zeros	min	median
age	500	0%	37.88	12.86	0%	17	37
education_num	500	0%	10.17	2.34	0%	2	10
capital_gain	500	0%	757.35	5,025.36	92.6%	0	0
capital_loss	500	0%	100.19	440.42	95%	0	0
hours_per_week	500	0%	41.21	12	0%	5	40

Categorical Features (9)

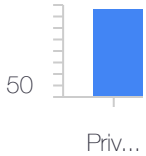
Chart to show

Standard

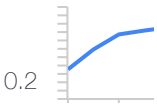
log expan

	count	missing	unique	top	freq top	avg str len
workclass	500	0%	6	Private	381	8.14
education	500	0%	15	HS-grad	177	8.57

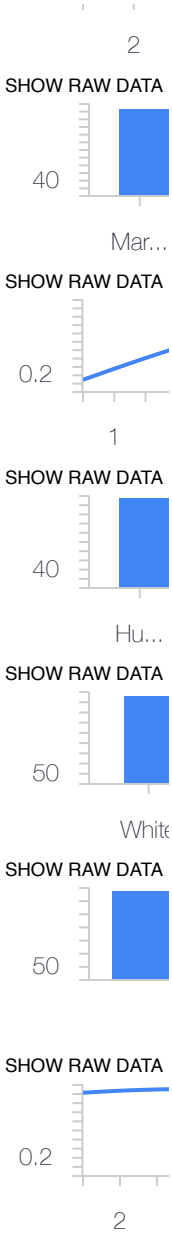
SHOW RAW DATA



SHOW RAW DATA



marital_status	500	0%	6	Married-…	226	14.25
occupation	500	0%	13	Adm-cle…	73	13.08
relationship	500	0%	6	Husband	195	9.15
race	500	0%	5	White	426	5.6
sex	500	0%	2	Male	338	4.65
native_country	500	0%	22	United-S…	457	12.49



9 - Freeze the schema

In [23]:
```
schema_path = "output/schema.pbtxt"
tfdv.write_schema_text(schema, schema_path)
print(f"Saved schema to: {schema_path}")
```

```
Saved schema to: output/schema.pbtxt
```

Checking if the file was saved

In [24]:
```
import os
print(os.getcwd())
print(os.path.abspath("output/schema.pbtxt"))
print(os.path.exists("output/schema.pbtxt"))
```

```
/content
/content/output/schema.pbtxt
True
```

In [25]:
```
!head -n 40 output/schema.pbtxt
```

```
feature {
  name: "age"
  type: INT
  presence {
    min_fraction: 1.0
    min_count: 1
  }
  skew_comparator {
    infinity_norm {
      threshold: 0.03
    }
  }
  drift_comparator {
    jensen_shannon_divergence {
      threshold: 0.1
    }
  }
  shape {
    dim {
      size: 1
    }
  }
}
feature {
  name: "workclass"
  type: BYTES
  domain: "workclass"
  presence {
    min_fraction: 1.0
    min_count: 1
  }
  shape {
    dim {
      size: 1
    }
  }
}
feature {
  name: "education"
  type: BYTES
```

In [26]:
```
from google.colab import files
files.download("output/schema.pbtxt")
```